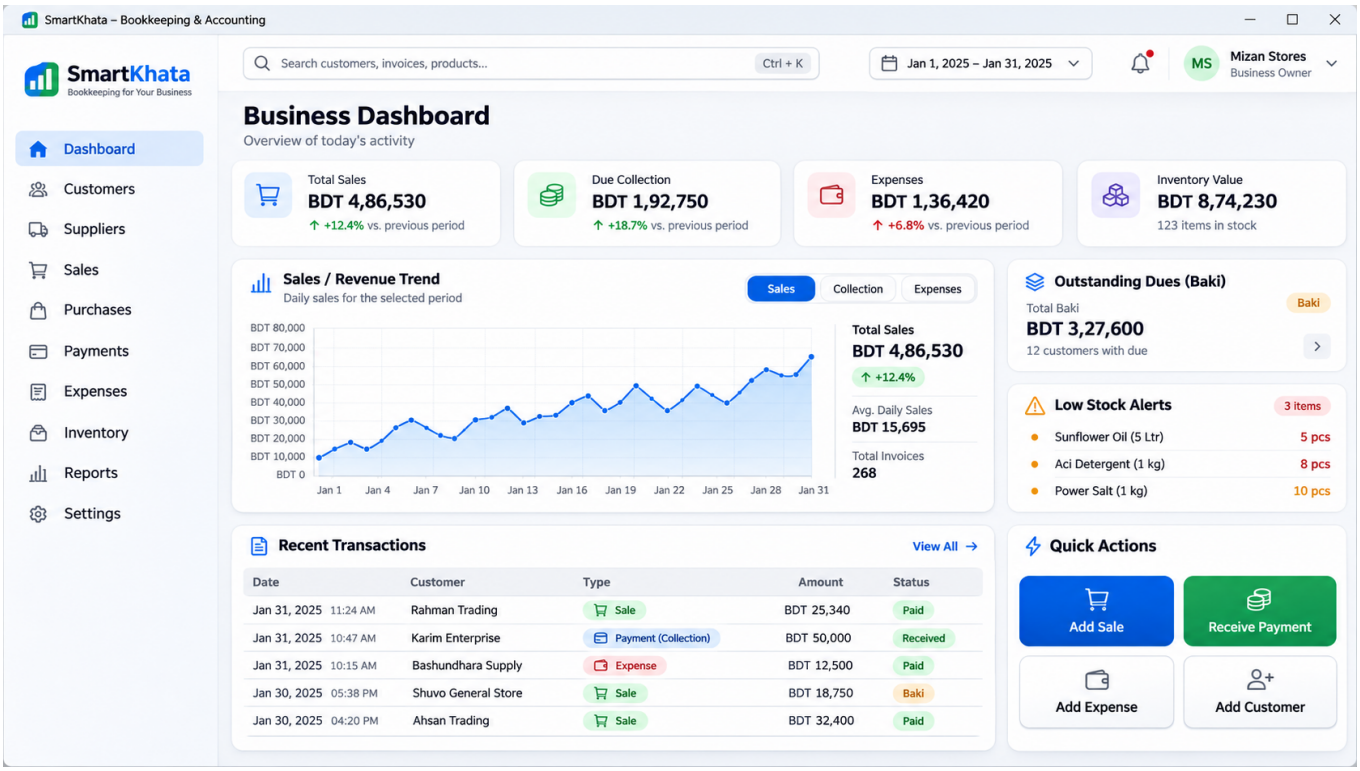
DESKTOP APP DEVELOPMENT / COPY-PASTE PROMPT PACK

SmartKhata

18 Google Antigravity prompts to build a full offline JavaFX + SQLite business accounting desktop application

JavaFX + FXML + AtlantaFX | **SQLite** local database | **Windows** standalone installer



Dashboard reference generated earlier for SmartKhata. Use the actual PNG file in the Antigravity project.

Includes: complete staged prompts, acceptance criteria, financial correctness rules, offline deployment requirements, and optional Nano Banana asset prompts.

How to use this PDF

Set up these files in the root of your Antigravity workspace before starting Prompt 1:

- **docs/SRS.md** - paste your full SmartKhata SRS from the conversation.
- **docs/dashboard-reference.png** - save the generated SmartKhata dashboard UI image from the conversation.
- **docs/implementation-status.md** - created/maintained by Antigravity to track tested work.

Run the prompts in order

Give Antigravity one numbered prompt at a time. Let it inspect the repository, implement only that stage, run actual checks, and explain any failures. Review the application yourself before moving to the next stage. The prompts deliberately preserve earlier work rather than restarting the project.

Key decisions made for your project

- **Desktop, not mobile:** JavaFX replaces the Android UI described in the SRS.
- **Offline-first:** SQLite is embedded locally. No remote database server or internet is required for normal work.
- **Local login:** offline credential setup replaces OTP/SMS login for version 1.
- **Exact money:** store BDT amounts as integer paisha or carefully controlled exact decimals, not floats.
- **Historical integrity:** posted transactions are corrected through auditable reversals or adjustments.
- **Visual reference:** recreate the screenshot as live JavaFX controls; do not use it as a flattened background image.

What “complete” means

An agent can generate code, but a successful result still requires human review and real tests. Prompt counts are not guaranteed: plan on these 18 implementation prompts plus debugging or design refinements when necessary. Antigravity may not be able to validate a Windows installer unless it is running on Windows.

Note: The Nano Banana prompts at the end are optional. JavaFX can draw most cards, charts, and icons natively; images are not required to implement the dashboard.

Prompt roadmap

The first five prompts establish a working foundation; the middle prompts implement the accounting workflows; the last prompts handle quality, recovery, and packaging.

#	Module	Deliverable
01	**Project architecture**	SRS mapping, offline adaptations
02	**JavaFX foundation**	Maven app that launches
03	**SQLite persistence**	Schema, migrations, transactions
04	**Desktop dashboard**	Match reference UI
05	**Local sign-in**	Owner account, business setup
06	**Customers**	CRUD, ledger, opening balance
07	**Suppliers**	CRUD, payable ledger
08	**Inventory**	Products, stock movements
09	**Sales**	Invoices, credit, stock deduction
10	**Purchases**	Supplier dues, stock receipts
11	**Payments**	Collections, allocations, audit
12	**Expenses/income**	Category tracking, no duplicates
13	**Dashboard data**	Real KPIs, chart, alerts
14	**Reports**	PDF/CSV exports
15	**Preferences**	Theme, language, settings
16	**Backup & restore**	Safe SQLite recovery
17	**QA and reconciliation**	Accounting + UI tests
18	**Windows release**	Runtime + installer + manual

PROMPT 01 / PLANNING

Project architecture and SRS mapping

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

You are the lead Java desktop architect for SmartKhata, an offline bookkeeping application for small businesses in Bangladesh.

INPUTS: Read docs/SRS.md and inspect docs/dashboard-reference.png. Treat the SRS as the functional source of truth and the image as the visual target. If either file is missing, identify it clearly before implementation; do not pretend it was inspected.

TECHNICAL BASELINE: Native JavaFX desktop UI (FXML + JavaFX CSS + AtlantaFX), Java 21 LTS, Maven, SQLite via JDBC, MVC / Service / DAO design, jpackage for Windows delivery. No React, Electron, WebView-based dashboard, Spring Boot backend, MySQL, or compulsory internet service.

TASK: Produce a detailed implementation plan only. DO NOT write application code yet.

DELIVERABLES:
1. Map each relevant SRS requirement (FR-01 through FR-23 and NFRs) to a desktop module and milestone. Flag cloud-only requirements for offline adaptation.
2. Explain local sign-in instead of OTP; local backups instead of cloud backups; disable remote admin, SMS, and sync until optional future phases.
3. Define presentation, controller, service, DAO, model, database, and reporting layers with package responsibilities and dependencies.
4. Define the main scenes, dialogs, navigation flow, and reusable JavaFX UI components needed to match the screenshot.
5. Propose a relational data model for business, users, customers, suppliers, products, sales, purchases, payments, ledger entries, stock movements, expenses, settings, and audit events.
6. Define accounting invariants: exact integer paisha or BigDecimal arithmetic, correct due calculations, no double counting, single-transaction updates, immutable/auditable adjustments, and idempotency.
7. Produce a traceable 18-phase task checklist, risks, decisions needing user review, and build/test strategy.

ACCEPTANCE: Written architecture, database outline, screen inventory, prioritized SRS mapping, and implementation checklist are present in docs/. No application code is generated in this phase.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 01 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 02.

PROMPT 02 / FOUNDATION

Bootstrap the Maven + JavaFX project

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement only the SmartKhata project skeleton based on the approved architecture in docs/ and the repository state.

TASK: Create a runnable native JavaFX + Maven desktop application using a compatible Java 21 LTS, JavaFX, AtlantaFX, SQLite JDBC, and JUnit setup. Use pinned compatible stable versions and document exact versions in the build file. Separate source packages for app/bootstrap, model, dao, service, controller, ui, database, util, and testing. Store FXML, CSS, fonts, icons, and images under src/main/resources.

IMPLEMENT:

1. Configure pom.xml, JavaFX Maven plugin, resources, test runner, and an unambiguous launch class.
2. Add an application bootstrap that initializes application directories, structured logging, and clean shutdown before opening the main window.
3. Add a basic styled shell (placeholder content is acceptable for this phase), global typography, CSS variables where JavaFX supports them, and primary window icon.
4. Add startup error handling with a user-friendly dialog and detailed local developer log; never log secrets.
5. Document run commands, required JDK, IDE import, and how to launch with mvn javafx:run or the configured task.
6. Keep UI responsive and avoid blocking the JavaFX Application Thread.

ACCEPTANCE: Clean checkout builds, automated sample test passes, and a JavaFX window launches. No database server, internet dependency, or browser window is required.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:

- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 02 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 03.

PROMPT 03 / DATA FOUNDATION

SQLite schema, migrations, and persistence

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement the persistent local SQLite data layer in the existing SmartKhata project. Do not replace the architecture from prior phases.

TASK: On first launch, create a writable database under the per-user application-data location, not the installed application folder or inside the JAR. Use a location equivalent to %LOCALAPPDATA%/SmartKhata/data/smartkhata.db on Windows. Allow safe development configuration overrides.

DATABASE REQUIREMENTS:
1. Versioned, repeatable schema upgrades (PRAGMA user_version or a clearly documented migration tool). Never drop existing user tables during an upgrade.
2. Core tables and foreign keys for business, users, memberships/roles, customers, suppliers, product categories/products, sales/sale items, purchases/purchase items, payments/payment allocations, customer/supplier ledger entries, expenses, other income, inventory movements, settings, and audit events.
3. All financial values must use integer minor units (BDT paisha) OR consistent exact DECIMAL via application BigDecimal conversion. Never store floating-point currency.
4. Unique business-scoped record identifiers and indexes for common customer, date-range, and transaction queries.
5. PRAGMA foreign_keys=ON for every connection; define busy timeout and appropriate WAL/journal strategy; handle locks and IO failures clearly.
6. Prepared statements and DAO interfaces; no SQL concatenation of user input.
7. A shared transaction boundary/service so sale + ledger + stock modifications can commit or roll back together.
8. Add repository integration tests using temporary SQLite files; test restart persistence, constraints, duplicate/idempotency protection, and migration from an earlier version.

ACCEPTANCE: Empty install auto-creates database; repeated launch keeps data; failed transaction rolls back; tests prove persisted values survive reopening.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 03 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 04.

PROMPT 04 / VISUAL DESIGN

Pixel-close dashboard UI from the reference

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Build the main SmartKhata desktop dashboard using native JavaFX FXML, AtlantaFX, custom JavaFX CSS, and reusable controls.

VISUAL REFERENCE: Inspect docs/dashboard-reference.png. Match its overall desktop composition closely, without rasterizing controls or embedding a web page. The reference contains a modern light dashboard with a sidebar, search toolbar, cards, line chart, table, and right-side utility panels.

IMPLEMENT:
1. Desktop target around 1600x900 with graceful use at 1280x800. Sidebar approximately 230-250 px; top toolbar; scrollable main area when necessary.
2. Left navigation: Dashboard, Customers, Suppliers, Sales, Purchases, Payments, Expenses, Inventory, Reports, Settings. Active state and accessible hover/focus states.
3. Header: business name/avatar, global search field shell, date-range controls, notification indicator.
4. Summary cards: Total Sales, Due Collection, Expenses, Inventory Value, with BDT labels and meaningful empty states.
5. Large revenue trend line chart card plus recent transactions table.
6. Right cards: Outstanding Dues (Baki), Low Stock Alerts, Quick Actions for Add Sale, Receive Payment, Add Expense, Add Customer.
7. Match white/slate backgrounds, blue/green accents, subtle gray strokes, rounded panels, icons, and clear 9-14 pt UI typography.
8. Define reusable component styles (cards, headings, tables, pills, dialogs, input fields, navigation items, primary/secondary buttons).
9. Wire nav buttons to a reusable view router; placeholder screens are allowed for unbuilt modules but must be clearly identified.
10. Use data-service interfaces for KPI values. Do not ship hard-coded fake accounting numbers in production code. Optional sample data belongs only in an explicitly separate demo/test mode.

ACCEPTANCE: App compiles and launches; sidebar navigation works; screenshot comparison notes show implemented and pending design differences; no dead visual buttons are represented as complete features.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 04 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 05.

PROMPT 05 / IDENTITY

Local owner login and business setup

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement offline-friendly SmartKhata account setup and login. Use the existing SQLite database, JavaFX scene system, and service/DAO boundaries.

DESKTOP ADAPTATION: The original SRS mentions mobile-number OTP. Replace it for the offline desktop MVP with local owner creation and a password or PIN-based login. No external SMS, cloud account, or internet is required.

IMPLEMENT:
1. First-run wizard: business name, owner name, optional phone/address, preferred currency BDT, interface language preference, and secure owner credential.
2. Subsequent startup: login screen; session lifecycle; logout and lock actions.
3. Store password/PIN hashes using a well-reviewed adaptive hashing algorithm with unique salts, never plaintext or reversible encryption.
4. Apply attempt throttling, clear errors, and privacy-preserving logs.
5. Keep current business/user context available to service-layer authorization checks; create authorization helpers for future staff roles without implementing cloud administration.
6. Business profile editing with validations and safe database updates.
7. Define a clearly explained offline account-recovery strategy that does not allow silently bypassing the credential and does not destroy books by default.
8. Add tests for first-run initialization, login success/failure, restarts, and unauthorized access.

ACCEPTANCE: A new user can initialize business and sign in fully offline, while invalid credentials do not open private records.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 05 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 06.

PROMPT 06 / CUSTOMERS

Customer management and customer ledger

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement complete customer management for SmartKhata, using persistent SQLite data and the shared desktop design system.

FEATURES:
1. Customers page with search, sortable table, Add Customer dialog, Edit Customer dialog, archive action, and customer detail view.
2. Fields: name (required), phone (optional), address, notes, created/updated timestamps, and business scope. Validate inputs and handle duplicate-looking records without unsafe silent merges.
3. Customer ledger view showing dates, transaction reference, debit/credit reason, amounts, and running outstanding balance.
4. Support a verified opening receivable balance by recording it as a traceable opening ledger entry rather than an editable detached total.
5. Provide an aggregate customer outstanding total for dashboard use.
6. Never delete customers with linked financial records; archive them instead and retain historical visibility.
7. Enforce owner/staff authorization via existing service guards, even before advanced staff UI exists.
8. Use pagination or virtualized TableView when records grow and show understandable empty states.
9. Add tests: create, edit, search, archive, opening balance, cross-business isolation, and ledger total correctness.

IMPORTANT: Do not invent sales or payment transactions; those will be added in later stages. Show a zero/empty ledger until legitimate financial records exist.

ACCEPTANCE: Customer records persist across restart; opening balances flow into ledger totals; archived customers remain available to historical reports.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 06 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 07.

PROMPT 07 / SUPPLIERS

Supplier management and payable ledger

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement supplier management and payable tracking with the same design language as the customer module.

FEATURES:
1. Supplier list, new supplier dialog, edit/archive, supplier detail, searchable records, and supplier-specific ledger.
2. Fields: supplier name, phone, address, contact person, notes, and business ID; record timestamps and validation.
3. Record traceable opening payable balances through supplier ledger entries.
4. Display outstanding supplier payables and transaction history, including future linked purchases and supplier payments.
5. Reuse customer design patterns without mixing customer receivables and supplier payables in one sign convention accidentally.
6. Retain historical supplier records when archived; enforce permissions and business data isolation.
7. Provide aggregate supplier payable service for dashboard widgets and reports.
8. Add supplier DAO/service/UI tests and ledger arithmetic tests.

IMPORTANT: Purchase entry itself is Prompt 10. Do not create placeholder purchase amounts as if they were real transactions.

ACCEPTANCE: Supplier records, opening balances, and payable totals survive restart and are isolated per business.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 07 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 08.

PROMPT 08 / INVENTORY

Products, inventory, and stock movements

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement products and inventory using the existing SQLite schema and JavaFX design system.

FEATURES:
1. Product list with search, category filter, stock column, unit, purchase cost, selling price, low-stock threshold, and active/archived states.
2. Add/Edit Product forms with optional SKU, category, unit (pcs/kg/litre/etc.), pricing, and validations.
3. Use a stock movement ledger (opening stock, purchase receipt, sale issue, manual adjustment, and reversal); do not silently overwrite current stock without an auditable movement.
4. Support manual stock adjustments only with a reason and acting user ID.
5. Expose availability checks and clear default behavior for insufficient stock (by default, reject a sale that would cause negative stock).
6. Provide inventory value with an explicitly documented cost method (e.g., weighted average or recorded current cost); use the same method in reporting.
7. Show a low-stock alert list and inventory summary for the dashboard.
8. Handle exact monetary arithmetic and meaningful quantity precision for units; document how fractional quantities work.
9. Test stock math, low-stock thresholds, duplicate adjustment retries, and archived products with history.

IMPORTANT: Sale/purchase posting will call stock services in future prompts. Keep stock update APIs transaction-safe.

ACCEPTANCE: Stock is derived consistently from movements; adjustments are auditable; reports and UI show accurate low-stock products.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 08 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 09.

PROMPT 09 / SALES

Sales and customer credit transactions

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Build end-to-end sales recording with invoice details, customer dues, and inventory updates. Preserve the existing architecture and use real database transactions.

FEATURES:
1. New Sale screen with customer selector (or walk-in sale), editable line items, quantity, unit price, discount, totals, and paid amount.
2. Support fully paid, partially paid, and credit sales. Outstanding receivable from a sale equals the invoice amount minus actual allocated payment.
3. Generate unique business-scoped invoice numbers and idempotency keys so repeated button clicks/retries do not double-post.
4. On confirmation, atomically save sale header/items, initial payment if any, customer receivable entries, and negative stock movements. Roll back all changes on any failure.
5. Enforce default no-negative-stock rule and product validation.
6. Provide invoice detail view, printable receipt preview, and readable status Paid / Partial / Baki. Use BDT currency formatting.
7. Implement correction with auditable void/reversal rules; never silently delete a posted sale or leave linked payments inconsistent.
8. Add sale list, filters, date range, and customer-level history integration.
9. Automated cases: BDT 1,000 all-credit sale creates BDT 1,000 due; BDT 1,000 sale with BDT 400 paid creates BDT 600 due; stock decreases once; failed posting rolls back; retry remains idempotent.

ACCEPTANCE: A legitimate sale affects invoices, customer dues, and inventory exactly once after application restart.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 09 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 10.

Purchases and supplier credit transactions

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement purchase recording and supplier payable management using the same financial integrity rules as sales.

FEATURES:
1. Purchase screen with supplier selector, item lines, quantities, unit costs, optional discounts, due date, and paid/partial/unpaid selection.
2. Generate business-scoped purchase reference number and prevent duplicate posting from retries.
3. On finalization, atomically save purchase header/items, supplier payable ledger entries, initial supplier payment if any, and positive inventory movements.
4. Reflect purchase costs in the inventory valuation method already selected; document treatment of purchase-related expenses.
5. Show purchase history, supplier filter, purchase detail view, and remaining unpaid amount.
6. Add controlled void/reversal mechanism preserving audit history and correcting stock/payables safely.
7. Prevent editing completed purchases in ways that silently change historical cost or posted stock. Provide reversal plus a corrected transaction instead.
8. Unit/integration tests for payable arithmetic and stock receipts, including BDT 750 purchase with BDT 250 paid -> BDT 500 payable.

ACCEPTANCE: Purchasing increases stock and creates accurate supplier dues without double counting or partial commits.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 10 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 11.

Customer collection and supplier payments

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement the complete payment/collection module, integrated with existing sales, purchases, and ledgers.

FEATURES:
1. Receive Payment flow for customer dues and Make Supplier Payment flow for supplier payables.
2. Select party, display current total outstanding, enter amount, date, method (cash, bank, mobile financial service, other), reference, and notes.
3. Support partial payment; define deterministic allocation to specific invoices or explicit oldest-first allocation. Persist payment-allocation links.
4. If amount exceeds a due balance, either reject with an understandable error or explicitly record an advance balance. Never silently discard the excess.
5. Update relevant ledgers in one atomic database transaction; prevent duplicate submissions via unique payment references/idempotency tokens.
6. Handle payment correction as a reversal or offsetting entry with an audit trail; do not delete old payments.
7. Payment history screen with filters, receipts, status, and source transaction linking.
8. Avoid double-counting a deposit received during sale creation as a second standalone collection.
9. Tests: BDT 1,000 due then BDT 400 collection -> BDT 600 due; supplier BDT 500 due then BDT 200 payment -> BDT 300 payable; retry does not duplicate payment; advance behavior is explicit.

ACCEPTANCE: Every payment is traceable, party balances reconcile, and historical reports remain stable.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 11 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 12.

PROMPT 12 / CASH TRACKING

Expenses and other business income

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement business expense and other-income entry, excluding income/expenses already accounted for in sales or purchases.

FEATURES:
1. Expense list + Add/Edit flow with amount, category, date, payment method, optional vendor/note/reference, and actor.
2. Other Income list + entry for business income not recorded as a sale (for example, a separate service receipt).
3. Category management and configurable defaults such as rent, utilities, salaries, transport, maintenance, and other.
4. Reconcile payments/expense totals without counting customer collections as new sales revenue or purchase expense as the same cost twice.
5. Use exact BDT monetary arithmetic; record adjustments/reversals instead of silently rewriting posted historical entries.
6. Add date filters, totals, validation, empty states, and meaningful inline success/error messages.
7. Provide consistent aggregate query methods for dashboard and reporting.
8. Tests: record BDT 120 utility expense and verify report shows it once; update/reverse through audit process; re-run date filters and restart persistence.

ACCEPTANCE: Expenses and other income persist, categorize properly, and do not corrupt sales, cash receipts, purchases, or due balances.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 12 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 13.

PROMPT 13 / DASHBOARD INTEGRATION

Real data dashboard, charts, and global search

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Replace any remaining dashboard placeholder metrics with actual local SQLite calculations and connect dashboard actions to working screens.

FEATURES:
1. Total Sales for selected date range, Due Collection for selected date range, Expenses for selected date range, and Inventory Value from current stock and defined valuation method.
2. Outstanding Dues panel showing customer receivables and count of customers with positive unpaid balances; label the as-of date explicitly.
3. Low-stock widget from inventory movements and thresholds; clicking an alert opens the product.
4. Daily sales line chart using valid posted sale totals; support sales/collections/expenses toggle without mixing their definitions.
5. Recent Transactions table combining approved transaction types with correct date/time, party, amount, and status.
6. Working Quick Actions opening Add Sale, Receive Payment, Add Expense, Add Customer.
7. Date range selector triggers efficient re-query; search navigates to matching customers, products, suppliers, or transactions as permitted.
8. Handle empty database gracefully (real zeros and friendly first-action guidance); no fictitious sales in a production database.
9. Use background Task/Service for slow queries and update JavaFX UI only on the FX Application Thread; avoid resource leaks.
10. Verify counts, totals, and visible labels against underlying SQLite queries, including a controlled demo data set used only in tests.

ACCEPTANCE: Dashboard changes correctly after posting a sale, payment, purchase, expense, or stock adjustment, and refreshes after reopening the app.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 13 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 14.

PROMPT 14 / REPORTING

Reports, exports, and ledger statements

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement the reporting module as an accurate and usable extension of the completed financial data layer.

REPORTS:
1. Daily/monthly/date-range sales, purchase summary, customer outstanding, supplier outstanding, customer ledger statement, supplier ledger statement, payments collected/paid, expense summary, other-income summary, and current inventory/low-stock report.
2. Define all reporting logic in reusable service/query classes; share the same ledger and reporting definitions used by the dashboard.
3. Report filters: date range, customer, supplier, category, status, and business as relevant. Clearly mark period totals versus current outstanding snapshots.
4. Export to a readable PDF with business header, dates, BDT currency, subtotals, and page numbers. CSV export must handle escaping, delimiters, and Unicode.
5. For PDF use an appropriate maintained Java library (e.g., JasperReports or OpenPDF) and document the dependency and template decisions.
6. Provide print preview/print action if supported by platform; display understandable errors on unavailable printer or destination.
7. Verify arithmetic correctness independently on sample posted data, including partially paid invoices and reversals.
8. Export only authorized business data and avoid disclosing sensitive credentials or technical logs.

ACCEPTANCE: Screen values reconcile with PDF/CSV exports and direct database queries for at least five representative accounting cases.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 14 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 15.

PROMPT 15 / POLISH

Settings, language, and preferences

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Complete the native desktop settings and user preferences area using the established design system.

FEATURES:
1. Business profile editor: business name, owner contact, address, optional logo, preferred invoice header, and BDT display preferences.
2. User preferences: theme (light default matching reference), date format, number format, table row size, low-stock threshold defaults, and report default period.
3. Prepare an internationalization resource-bundle system. Provide English labels everywhere; include Bangla localization for core navigation, key actions, and transaction labels where possible, with proper Bengali font rendering and fallback.
4. Local notifications for due reminders and low-stock alerts inside the app. No mandatory SMS/internet integration.
5. Clearly show data storage location, app version, legal/privacy notice, and Backup / Restore entry points.
6. Persist preferences to SQLite; validate them and apply without corrupting accounting data.
7. Add keyboard navigation, tab order, tooltips, confirm dialogs, and accessible error/empty-state patterns.
8. Never use settings as a shortcut to edit historical financial ledger entries directly.

ACCEPTANCE: Settings survive restart, UI remains usable at smaller desktop sizes, and Bangla characters render where translations are supplied.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 15 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 16.

PROMPT 16 / DATA SAFETY

Reliable SQLite backup and recovery

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Implement offline backup and restore for SmartKhata, paying special attention to SQLite consistency and accidental data loss.

REQUIREMENTS:
1. A user-facing Back Up Now action that selects an output location and produces a coherent database backup with visible time, size, and completion status.
2. Use the SQLite online backup API, VACUUM INTO where appropriate, or an equivalent transactionally safe approach. Do NOT blindly copy an active SQLite file when WAL files may contain uncheckpointed transactions.
3. Add a database integrity check and metadata manifest (app/schema version, timestamp) for restored backups.
4. Restore workflow must warn the user, create a safety backup of the currently active database, close active connections, validate candidate backup, restore atomically, then relaunch or reinitialize safely.
5. Protect against restore from incompatible schema versions and corrupt/untrusted files; never overwrite silently.
6. Document where the local database lives and how users should store backups safely, ideally on a separate drive.
7. Optional: prompt to back up after a configurable interval; no external cloud dependency.
8. Test backup/restore across restart and populated datasets, with deliberate corruption/error-path tests.

ACCEPTANCE: A fresh installation can restore a valid backed-up business; balances, records, and inventory match the original; invalid backup does not destroy current data.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 16 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 17.

PROMPT 17 / QUALITY ASSURANCE

Automated tests, accounting audit, and UI QA

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Act as the QA lead and senior Java desktop engineer. Audit the existing SmartKhata implementation against docs/SRS.md, docs/implementation-status.md, and the design reference. Do not introduce unrelated new features.

TEST PLAN:
1. Run unit, SQLite integration, migration, DAO/service, and JavaFX UI smoke tests that are feasible in the current environment. Report commands and actual results.
2. Establish a deterministic test dataset and verify credit sale BDT 1,000 -> due 1,000; receive BDT 400 -> due 600; supplier credit purchase BDT 750 with payment 250 -> payable 500; expense BDT 120 displayed exactly once.
3. Verify stock decreases on sale and increases on purchase once, rejects unavailable stock by default, and reversals preserve historical audit.
4. Test retrying the same transaction or payment, app restart, database file permissions, incorrect credentials, archived parties, duplicate-looking customers, and empty-state flows.
5. Test concurrent/rapid button clicks, long report queries, SQLite locked database behavior, validation errors, and rollback after a simulated failure.
6. Check business/data isolation in DAO queries; SQL injection resistance; no secrets in logs; correct CSV escaping.
7. Compare main UI layout against docs/dashboard-reference.png at desktop target and minimum window sizes. Fix clipping, alignment, typography, disabled states, and overflow.
8. Produce a requirement-by-requirement test matrix (pass/fail/not implemented) and a prioritized list of remaining issues.

ACCEPTANCE: The app compiles, critical accounting invariants pass, blocker bugs are fixed, and unresolved items are explicitly documented.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Stage 17 checkpoint: Confirm its acceptance criteria in a running application or actual test output before starting Prompt 18.

PROMPT 18 / SHIPPING

Windows installer and release checklist

Paste the prompt below into the same Antigravity workspace, after the previous stage is accepted.

Package SmartKhata as a standalone Windows desktop app with an embedded Java runtime, preserving local SQLite files across upgrades and uninstalls as documented.

TASK:
1. Verify Maven reproducible build and unit/integration test execution.
2. Configure jlink/jpackage or an appropriate Java packaging pipeline for JavaFX and SQLite JDBC, with required native libraries included. Do not assume packaging works until tested.
3. Produce a Windows .exe/.msi installer from a compatible Windows build environment; if current environment is not Windows, prepare a reproducible Windows packaging script/config and explicitly state that actual Windows installer testing was not performed.
4. Set installer/product name SmartKhata, version, icon, publisher placeholder (clearly marked), application shortcuts, and appropriate file locations.
5. Ensure new installs create %LOCALAPPDATA%/SmartKhata/data/smartkhata.db on first launch; upgrades do not overwrite or migrate destructively; uninstall behavior does not surprise users about data retention.
6. Validate all screens and exports on a clean Windows machine without requiring XAMPP, MySQL, PostgreSQL, internet, or separately installed JDK/JRE.
7. Add a small user manual: first launch, add customer, credit sale, collect payment, backup, restore, and data file location.
8. Generate a release checklist with app size, build commands, third-party license notices, known limitations, backup recommendations, and versions.
9. Run a smoke scenario using real local persistence, shut down, relaunch, and verify data was retained.

ACCEPTANCE: Installer is generated and validated on Windows where possible; app functions fully offline; user business data survives closing and reopening; all untested release steps are clearly labeled.

STANDARD HANDOFF INSTRUCTIONS

COMPLETION PROTOCOL:
- Read the existing repository before editing. Preserve working code, migrations, and data.
- Implement only this stage and the smallest required dependencies. No unrelated rewrites.
- Run the relevant compile, tests, and application launch checks available in the environment; never claim a test passed if it was not run.
- Update docs/implementation-status.md with files changed, commands/results, requirement IDs covered, known limitations, and exact next steps.
- Stop after this stage and report completion; do not silently proceed to the next prompt.

Final checkpoint: Verify a clean install and a successful offline persistence smoke test before distributing the application.

Optional Nano Banana visual asset prompts

You do not need external artwork for JavaFX charts, tables, buttons, or navigation. Use these only if you want branded assets. Do not ask an image generator to create the actual interactive application interface.

Optional Nano Banana Prompt A

Asset: App logo and Windows icon

Create one original professional logo concept for a Bangladesh-focused offline desktop bookkeeping product named SmartKhata. Build a recognizable icon combining a ledger/book and an upward bar chart in a simple blue (#1358E7) and emerald (#16A36C) palette. Clean vector-like geometry, no gradients required, no complex tiny details. Generate a transparent-background square logo suitable for Windows icon use at 16, 32, 64, and 256 pixels. Avoid any imitation of the TallyKhata logo or other brands. Deliver the symbol plus a separate full-size horizontal wordmark variant if multiple outputs are supported. Use legible English text only for the wordmark. Do not place the logo on a screenshot or decorative mockup.

Optional Nano Banana Prompt B

Asset: Empty-state illustration

Create a minimal original vector-style illustration for an offline business bookkeeping desktop app's empty dashboard: a friendly open ledger, small calculator, and a few simple receipt shapes. Transparent background, blue/emerald/gray palette, soft rounded geometry, professional modern SaaS feel, no people, no written text, no decorative scene background, no real business information, no logos. Design to sit cleanly inside a 440x240 JavaFX empty-state panel. Keep outlines and shapes simple enough to look sharp after scaling.

Optional Nano Banana Prompt C

Asset: Onboarding welcome illustration

Create a tasteful original illustration for the SmartKhata JavaFX desktop first-run welcome screen, showing a small shop counter with ledger book, currency receipts, and inventory boxes, with a subtle modern flat vector aesthetic. Bangladesh small-business context without flags, trademarks, or brand logos. Use a light neutral background or transparent background, with blue and green accent palette. No text and no human faces. Wide 4:3 composition with generous whitespace for adjacent onboarding form. Avoid copying any reference application's artwork.

How to integrate generated images

- Export PNG/SVG assets (when available) in appropriate sizes. Use transparent backgrounds for logos and empty-state illustrations.
- Put approved visuals in src/main/resources/assets/ and load them from the classpath, not a machine-specific absolute file path.
- Provide PNG raster assets for JavaFX ImageView where needed; convert approved SVGs to suitable PNG/icon resources when native support is not available.
- Keep meaningful application text, buttons, tables, and charts native JavaFX controls so they remain interactive and accessible.

Release acceptance checklist

Use this list after Prompt 18, before distributing SmartKhata to a user.

□ Launches on a clean Windows computer without installing MySQL, XAMPP, PostgreSQL, or a separate JRE.

□ All business records remain after application close and reopen.

□ Database lives in the user profile/application-data directory, not the installed program directory.

□ A BDT 1,000 credit sale followed by BDT 400 collection results in BDT 600 outstanding due.

□ Purchases correctly increase inventory and unpaid supplier balances.

□ Payments cannot post twice from a repeated click or retry.

□ Reversing completed financial entries retains an audit trail and reconciles totals.

□ Dashboard KPI values match transaction history and reporting services.

□ CSV/PDF reports show correct selected date ranges and monetary values.

□ Backup can be restored successfully on a fresh install without data corruption.

□ Desktop UI remains usable at 1280x800 and visually resembles the reference.

□ No production financial totals are hard-coded; empty businesses display honest zero/empty states.

□ Build and smoke-test results are recorded with any incomplete items disclosed.

Final delivery artifacts

- Source code repository and README
- Database migration scripts and automated tests
- Windows installer or reproducible Windows packaging instructions
- End-user manual including backup/restore steps
- Updated SRS coverage and known limitations report

SmartKhata - project-specific prompt guide. All prompts are intended to be edited as architecture and implementation findings evolve.